Assignment 07

Published | Assign To | Edit

Learning Objectives

The goal of this assignment is to learn about DNS, implement logging, metrics instrumentation, and collection.

Domain Name

Namecheap offers a free domain with the GitHub Student Developer pack, but only `.me` *TLD domain.*

1. Register a domain name with a domain registrar such as **Namecheap (https://www.namecheap.com/)**, **AWS (https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/domain-register.html)**, **gandi.net (https://www.gandi.net/en-US)**, or **GoDaddy (https://www.godaddy.com/)**.
2. Your domain name can be anything.

Infrastructure as Code w/Terraform

IAM Roles & Policies Updates

Warning

- The assignment objective requires you to identify the permissions needed and create appropriate roles and policies.
- Not all required roles and policies are listed in this assignment or on the lecture page.

Application Logging & Metrics

Reference Documentation:

1. CloudWatch Agent
    1. **Unified CloudWatch Agent (https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/UseCloudWatchUnifiedAgent.html)**
2. CloudWatch Logs
    1. **Create IAM Roles and Users for Use With CloudWatch Agent (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/create-iam-roles-for-cloudwatch-agent.html)**

2. **Install CloudWatch Agent Package on an Amazon EC2 Instance (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/install-CloudWatch-Agent-on-first-instance.html#download-CloudWatch-Agent-on-EC2-Instance-first)**

3. CloudWatch Metrics
    1. **Retrieve Custom Metrics with StatsD (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-custom-metrics-statsd.html)**
    2. **Retrieve Custom Metrics with collected (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-custom-metrics-collectd.html)**

IAM Users, Roles & Policies

Add IAM roles & policies needed to meet the assignment objectives to the Terraform template.

AMI Updates

1. Update your packer template to install the Unified CloudWatch Agent in your AMIs.
2. Your CloudWatch agent must be set up to start automatically when an EC2 instance is launched using your AMI.

IAM Updates

1. Update Terraform to attach the IAM role to the EC2 instance for use with CloudWatch Agent.

Userdata Script Updates

1. Your user data script should configure the Cloudwatch agent and restart it.

Web Application Updates

Metrics

1. As a user, I want all application log data to be available in CloudWatch.
2. As a user, I want metrics on API usage in CloudWatch.
3. Create following custom metrics for every API we have implemented in the web application. The metrics data should be collected in CloudWatch.
    1. `Count` number of times each API is called.
    2. Using `Timer` metrics data type, and time (in milliseconds) for each API call, so we can understand how long it takes for the application to process an API call.
    3. Using `Timer` metrics data type, time (in milliseconds) each database query executed by your application.
    4. Using `Timer` metrics data type, time (in milliseconds) each API call made to the AWS S3 service by your application.
4. You can retrieve custom metrics using either **StatsD (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-custom-**

metrics-statsd.html) or collectd (https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatch-Agent-custom-metrics-collectd.html) .

Logging

1. All user requests generate log messages that can be seen in CloudWatch
2. Log messages are meaningful and written in English with attention to grammar and spelling.
3. All errors and exceptions are logged with the appropriate log levels, and stack traces are included where applicable.

Submission Requirements

Step 1: Create Submission Folder

Create a folder using this naming convention:

```
firstname_lastname_neuid_a_##
```

Replace `##` with the assignment number (e.g., `01`, `02`, etc.)

Step 2: Clone Repositories

1. Navigate to your submission folder
2. Clone **all** required GitHub repositories using:

bash

```
git clone [repository-url]
```

3. **Important:** Use `git clone` to preserve your complete commit history and branch information

Step 3: Create and Verify ZIP File

1. Create a ZIP file of your submission folder:

```
firstname_lastname_neuid_a_##.zip
```

2. **Verification Step:** Extract your ZIP file in a different location and confirm:
   - All repositories are present
   - Commit history is intact
   - All required branches exist

Step 4: Submit

Upload your ZIP file to the assignment submission portal.

Important Notes

- **Multiple Submissions Allowed:** You may resubmit if needed. Only your latest submission will be graded.
- **Quality Check:** Always verify your ZIP file contents before final submission to avoid issues.
- **Complete Repositories Required:** Submit code from all repositories mentioned in the assignment requirements.

Troubleshooting

If you encounter issues with your ZIP file, feel free to resubmit after verification. Take time to ensure all requirements are met before your final submission.

Grading Guidelines for Cloud-Native Web Application

System Stability & Performance

- **No Server Errors**: Application must not generate 500 internal server errors during testing
- **Application Uptime**: No restart of the application server required between API calls
- **Error Handling**: Application should handle invalid requests gracefully with appropriate HTTP status codes

Demo Requirements

- **Platform**: Students must demonstrate the web application from their own laptops
- **API Testing**: APIs must be tested using REST clients (Postman, Restlet, Insomnia, etc.)
  - Browser-based testing is **not acceptable**
- **UI Restriction**: Application must **not** have any user interface components
  - Verify complete absence of HTML pages, forms, or visual components

Security Implementation

Authentication & Authorization

- **Authentication Method**: Must use Basic Authentication with a token-based approach
  - Session-based authentication is **not allowed**
  - JWT tokens are **not allowed**
- **Password Security**: All passwords must be encrypted using BCrypt with salt
  - Verify in the database that passwords are properly hashed
  - Check that salt is unique per password

Data Protection

- **Response Payload Security**: Password fields must **never** be included in API responses
  - Test all endpoints that return user data
- **Username Validation**: Only valid email addresses can be used as usernames
  - Test rejection of non-email username formats

Data Integrity & Business Logic

Account Management

- **Duplicate Prevention**: Application must prevent the creation of multiple accounts with the same email address
  - Test duplicate email registration scenarios
  - Verify appropriate error responses for duplicate attempts
- **Audit Fields Protection**: System-managed fields must be read-only
  - `account_created` field: Set automatically by application, never by user input
  - `account_updated` field: Set automatically by application, never by user input
  - Test attempts to manually set these values through API calls

Code Quality & Standards

Configuration Management

- **No Hard-Coding**: Variable values must be configurable (environment variables, config files)
- **Default Values**: Acceptable when overrideable during runtime or demos
- **Demo Flexibility**: Values must be modifiable when requested by the instructor/TA

Code Style & Documentation

- Follow industry-standard style guides for your programming language
- Include meaningful comments for complex logic
- Maintain consistent formatting and naming conventions
- Provide clear README files with setup and usage instructions

Application Stability & Error Handling

Runtime Stability

- **No Crashes**: Application must handle API calls without crashing
- **No Restarts**: Application/VM should not require restarts during demos (unless specifically required for feature demonstration)
- **Graceful Degradation**: Handle edge cases and invalid inputs appropriately

Error Management

- **No 500 Errors**: All server errors must be properly handled
- **Meaningful Error Messages**: Provide clear, actionable error responses
- **Input Validation**: Validate and sanitize all user inputs
- **Resource Management**: Properly handle database connections, file operations, and external API calls

Edge Case Handling

- Test boundary conditions (empty inputs, maximum values, null cases)
- Handle network timeouts and connection failures

- Implement appropriate fallback mechanisms
- Validate data integrity throughout the application lifecycle

Repository Content & Hygiene

File Exclusion Standards

- **IDE Files**: No IDE-specific configuration files (`.idea/` , `.vscode/` , `.eclipse/` , etc.)
- **Build Artifacts**: No compiled files or build outputs
  - Java: No `.class` , `.jar` , `.war` files
  - Node.js: No `node_modules/` directory
  - General: No `build/` , `dist/` , `target/` directories
- **System Files**: No OS-specific files (`.DS_Store` , `Thumbs.db` , etc.)

Dependency Management

- **External Dependencies**: Third-party libraries from package managers (Maven, npm, pip) must **not** be committed
- **Gitignore Configuration**: Verify `.gitignore` file is properly configured and comprehensive
  - Must include language-specific patterns
  - Must exclude build artifacts and dependencies
  - Should include common IDE and OS files

Grading Guidelines

Warning: The following guidelines are for information only and are subject to change at the discretion of the instructor and TA.

Common Grading Rubrics

Git

1. All commits to GitHub repositories must be made via pull request. No changes can be made by pushing commits directly to the main branch. The exception to this is the first commit made to the repository to initialize it, which can be done directly from the UI.
2. All pull requests must have status checks and be enforced. Merging pull requests that did not pass status checks may result in penalties.

Security

1. Committing any kind of credentials to the code or uploading them as part of your Canvas submission may result in a penalty.

2. Service providers may automatically disable your keys or suspend your account if they detect keys being committed in your GitHub repositories, even if they are private. This can severely affect your assignment grade and your ability to work on assignments in the course. This is not grounds for assignment extensions; late penalties will apply as documented in the syllabus.

Software Engineering Best Practices

1. Values for variables should not be hardcoded in the code. It's OK to set default values for variables, but if the value cannot be modified during the demo or when the TA or instructor asks, you may be penalized.
2. Follow industry standards and use a style guide for your code. You can find them with simple web searches.

Code Stability, Error Handling, & Edge Cases

1. The application should not crash between various API calls.
2. The application, VM, or other resources should not require a restart during the demo unless the grading guidelines require it to demonstrate a specific feature/functionality.
3. The server cannot respond with a 500 Internal Server Error. This usually highlights areas of code that do not have proper error handling.

Previous Assignment Objectives

TAs must verify that students have resolved all open issues from the previous assignment(s).

Git & GitHub Repository

Git Repository Content Check

1. Check the repository for any IDE-specific files. IDE configuration files must not be in the repository.
2. Verify their .gitignore configuration.
3. Check the repository for build artifacts such as `.class`, `.jar`, `.war` files and `build`, `node_modules`, `venv` directory. None of these should be checked into the repository.
4. Check for dependencies. Dependencies from the Maven repository, npm, or Python should not be committed to the git repository.

Git Forking Workflow

1. No direct commits are made to the organization's repository. Verify that the student works from a forked repository and uses the feature branch. There should be no direct commits to the main branch in their forked repository.

Application Logging & Metrics

Package AMI Updates

1. Verify the CloudWatch Unified Agent is installed in the AMI.
2. Verify the CloudWatch configuration is copied to the AMI.

CloudWatch IAM

1. Verify that the custom IAM role is created by Terraform and attached to the EC2 instance when the stack is created.
2. The custom role should be deleted when the Terraform stack is destroyed.
3. Note: No AWS credentials should be in the AMI/EC2 instance.

CloudWatch Agent Configuration

1. Userdata scripts configure the CloudWatch agent when the instance is launched.

Application Logs

1. The application generates meaningful log messages and writes them to a file.
2. Logs are available in CloudWatch in near real-time.
3. A new log group & stream is created for the stack.

Application Metrics

1. Make Requests to various APIs implemented and check for `counts` being published as custom metrics. Students must be able to navigate the AWS console without help from the TA to demo logs and metrics.

DNS

Domain Name & Zone Delegation for Sub-Domains

1. The student has registered a domain name or has an existing domain name to use in the course.

Points	10
Submitting	a file upload
File Types	zip
Allowed Attempts	3

Due	For	Available from	Until
Oct 30 at 3pm	Everyone	Oct 23 at 6pm	Oct 31 at 3pm

+ Rubric